A* Search

```
import heapq

initial = (2, 8, 3,
           1, 6, 4,
           7, 0, 5)

goal = (1, 2, 3,
        8, 0, 4,
        7, 6, 5)


def manhattan(state):
    distance = 0

    for i in range(9):
        tile = state[i]

        if tile != 0:
            current_row, current_col = divmod(i, 3)
            goal_pos = goal.index(tile)
            goal_row, goal_col = divmod(goal_pos, 3)

            distance += abs(current_row - goal_row)
            distance += abs(current_col - goal_col)

    return distance


def print_state(state):
    for i in range(0, 9, 3):
        print(state[i], state[i+1], state[i+2])
    print()


def successors(state):
    pos = state.index(0)
    r, c = divmod(pos, 3)

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
    for dr, dc in moves:
        nr, nc = r + dr, c + dc

        if 0 <= nr < 3 and 0 <= nc < 3:
            new_pos = nr * 3 + nc
            new_state = list(state)

            new_state[pos], new_state[new_pos] = \
                new_state[new_pos], new_state[pos]

            yield tuple(new_state)


def a_star():
    pq = []
    heapq.heappush(pq, (manhattan(initial), 0, initial))

    parent = {initial: None}
    cost = {initial: 0}

    while pq:
        f, g, current = heapq.heappop(pq)

        if current == goal:
            path = []

            while current is not None:
                path.append(current)
                current = parent[current]

            return path[::-1]

        for next_state in successors(current):

            new_g = g + 1

            if next_state not in cost or new_g < cost[next_state]:
                cost[next_state] = new_g
                h = manhattan(next_state)
                new_f = new_g + h

                parent[next_state] = current
                heapq.heappush(pq, (new_f, new_g, next_state))
```

```
    return None


path = a_star()

print("A* SEARCH - MANHATTAN DISTANCE")
print("-------------------------------")

for i, state in enumerate(path):
    print("Step:", i)
    print_state(state)
    print("g =", i)
    print("h =", manhattan(state))
    print("f =", i + manhattan(state))
    print()

print("Goal reached!")
print("Total moves =", len(path) - 1)
```

